MICROSAR TLS

Technical Reference

Transport Layer Security

Version 5.02.01

Authors	Thorsten Albers, Fabian Eisele, Alex Lunkenheimer
Status	Released

Document Information

History

Author	Date	Version	Remarks
Thorsten Albers	2010-10-28	1.00.00	Creation of document
Thorsten Albers	2011-01-12	1.01.00	Tls_Shutdown added
Thorsten Albers	2011-03-29	1.02.00	More detailed descriptions
Thorsten Albers	2011-08-26	1.03.00	ECC support included
Thorsten Albers	2011-10-27	1.04.00	minor updates
Thorsten Albers	2012-08-06	1.05.00	TLS 1.2 support added, cipher suites with SHA256 added
Fabian Eisele	2015-05-27	2.00.00	Added MSR4R12 parts
Thorsten Albers	2015-11-03	2.00.01	Conversion to new template, update of API descriptions, add some configuration parameter descriptions
Thorsten Albers	2016-03-01	2.01.00	Moved crypto calculation for handshake into separate MainFunction. Layout changed for new Vector logo.
Thorsten Albers	2016-03-07	3.00.00	FEAT-1347: TLS as TcpIp plug-in
Thorsten Albers	2016-04-20	3.01.00	FEAT-1741: Support TLS cipher suite TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256
Thorsten Albers	2016-05-23	3.01.01	FEAT-1741: Support server certificate validation using Certificate Revocation Lists (CRL)
Thorsten Albers	2016-10-27	4.00.00	FEAT-2005: Release of TLS_RSA_WITH_AES_128_CBC_SHA256, also support TLS1.2 with RSA and client certificate FEAT-1996: TLS Heartbeat Extension
Thorsten Albers	2017-02-28	4.00.01	Review integration
Thorsten Albers	2017-03-29	5.00.00	FEAT-2275: Release of TLS
Thorsten Albers	2017-05-16	5.01.00	STORYC-592: TLS Server CertChain Callout in TLS Client STORYC-638: Report Invalid Server CertChain STORYC-1506: Grant access to the TLS master secret
Thorsten Albers	2018-02-27	5.02.00	STORYC-4467: Adapt TLS to be compatible with TcpIp that only provids the ASR4.2 transmit API
Alex Lunkenheimer	2018-11-02	5.02.01	Review integration

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR_SWS_DET.pdf	4.2.1
[2]	AUTOSAR	AUTOSAR_SWS_DEM.pdf	4.2.1
[3]	AUTOSAR	AUTOSAR_BasicSoftwareModules.pdf	V1.0.0
[4]	AUTOSAR	AUTOSAR_SWS_SocketAdaptor.pdf	4.2.1
[5]	AUTOSAR	AUTOSAR_SWS_TcpIp.pdf	4.2.1
[6]	IETF	RFC 793: TCP	-
[7]	IETF	RFC 768: UDP	-
[8]	IETF	RFC 2246: TLS 1.0	-
[9]	IETF	RFC 5246: TLS 1.2	-
[10]	IETF	RFC 5280: Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile	-
[11]	IETF	RFC 6066: Transport Layer Security (TLS) Extensions: Extension Definitions	-
[12]	IETF	RFC 6960: X.509 Internet Public Key Infrastructure Online Certificate Status Protocol – OCSP	-

Scope of the Document

This technical reference describes the general use of the TLS basis software. Please refer to your Release Notes to get a detailed description of the platform (host, compiler) your Vector Ethernet Bundle has been configured for.



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1 Component History 7

2 Introduction 8
2.1 Architecture Overview 9

3 Functional Description 12
3.1 Features 12
3.1.1 Supported Cipher Suites 12
3.1.2 Support certificate validation using OCSP 12
3.1.3 Support certificate validation using CRL 12
3.1.4 Support TLS 'trusted_ca_keys' extension as specified in RFC 6066 13
3.1.5 TLS Server CertChain Callout in TLS Client 13
3.1.6 Limitations 13
3.1.6.1 L001: Client support only 13
3.1.6.2 L002: Message size 13
3.1.6.3 L005: TLS fragmentation 13
3.1.6.4 L007: Maximum number of certificates in the CRL 14
3.1.6.5 L008: Record Layer message size limit negotiation 14
3.2 Initialization 14
3.3 States 14
3.4 Main Functions 14
3.5 Error Handling 14
3.5.1 Development Error Reporting 14
3.5.2 Production Code Error Reporting 15

4 Integration 16
4.1 Scope of Delivery 16
4.1.1 Static Files 16
4.1.2 Dynamic Files 16
4.2 Critical Sections 16
4.3 General integration notes 17
4.3.1 External access to memory (e.g. XCP or Diagnostics) has to be limited 17
4.3.2 Access to real global time and data information 17
4.3.3 Access to real random values 17
4.3.4 CRL configuration 17
4.3.5 Heartbeat configuration 18
4.3.6 TLS Connection Setup 18
4.3.7 Dependency to TcpIp 18

4.3.8 Server CertChain Callout 19
4.3.9 Access to the TLS master secret 20

5 API Description 21
5.1 TLS usage via the TcpIp module 21
5.1.1 General functions 21
5.1.2 Standard communication functions 21
5.1.3 TLS specific functions 21
5.2 Type Definitions 21
5.3 Interrupt Service Routines provided by TLS 22
5.4 Services provided by TLS 22
5.4.1 Tls_InitMemory 22
5.4.2 Tls_Init 23
5.4.3 Tls_GetVersionInfo 23
5.4.4 Tls_GetNvmBlockIdForUsedRootCert 24
5.4.5 Tls_RootCertWasModified 25
5.4.7 Tls_MainFunction 26
5.4.8 Tls_MainFunctionLowPrio 26
5.4.11 Tls_SetClientCertInfo 28
5.4.12 Tls_Close 28
5.4.13 Tls_ProvideTxBuffer 29
5.4.14 Tls_TransmitTo 29
5.4.15 Tls_Received 30
5.4.16 Tls_GetMasterSecret 31
5.5 Services used by TLS 31
5.6 Callback Functions 33
5.6.1 Tls_TcpConnected 33
5.6.2 Tls_TcpIpEvent 33
5.6.3 Tls_RxIndication 34

6 Configuration 35
6.1 Configuration Variants 35
6.2 Configuration with DaVinci Configurator Pro 35

7 Glossary and Abbreviations 36
7.1 Glossary 36
7.2 Abbreviations 36

8 Contact 37

Illustrations

Figure 2-1 AUTOSAR 4.3 Architecture Overview 9
Figure 2-2 AUTOSAR 4.x Architecture Overview 10
Figure 2-3 Interfaces to adjacent modules of the TLS 10
Figure 2-4 Global TLS usage architecture 11
Figure 4-1 Get a socket for TLS and set buffer sizes 19
Figure 4-2 Bind and connect a socket 19

Tables

Table 1-1 Component history 7
Table 3-1 Service IDs 15
Table 3-2 Errors reported to DET 15
Table 4-1 Static files 16
Table 4-2 Generated files 16
Table 5-1 Type definitions 22
Table 5-2 Tls_InitMemory 23
Table 5-3 Tls_Init 23
Table 5-4 Tls_GetVersionInfo 24
Table 5-5 Tls_GetNvmBlockIdForUsedRootCert 24
Table 5-6 Tls_RootCertWasModified 25
Table 5-7 Tls_RootCertWasModified 25
Table 5-8 Tls_MainFunction 26
Table 5-9 Tls_MainFunction 26
Table 5-10 Tls_GetSocket 27
Table 5-11 Tls_ChangeParameter 27
Table 5-12 Tls_SetClientCertInfo 28
Table 5-13 Tls_Close 29
Table 5-14 Tls_ProvideTxBuffer 29
Table 5-15 Tls_TransmitTo 30
Table 5-16 Tls_Received 30
Table 5-17 Tls_GetMasterSecret 31
Table 5-18 Services used by the TLS 33
Table 5-19 Tls_TcpConnected 33
Table 5-20 Tls_TcpIpEvent 34
Table 5-21 Tls_RxIndication 34
Table 7-1 Glossary 36
Table 7-2 Abbreviations 36

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features
1.00.xx	Initial version
1.01.xx	Tool based configuration, ASR and BSD APIs
1.02.xx	NvM support, server certificate verification
2.00.xx	Reduction of function local crypto workspaces, support for ECC cipher suites
2.01.xx	Connection specific certificates
3.00.xx	Support of Ipv6 and Ipv4 address format
3.01.xx	Support for certificates with ECDSA-with-SHA256 signature
3.02.xx	Support of TLS 1.2 added
3.03.xx	Support Pass-Through Mode
3.04.xx	Support reading the NVM Block ID of the used root certificate
4.00.xx	API change for multi controller support
4.01.xx	Support certificate validation using OCSP stapling V1
5.00.xx	Support the new ASR4.2.1 TcpIp API, but the TLS API is still according to TcpIp ASR4.0.x
5.01.xx	Moved crypto calculation for handshake into separate MainFunction.
6.00.xx	FEAT-1347: TLS as TcpIp plug-in
7.00.xx	FEAT-1996: TLS Heartbeat Extension
7.01.xx	FEAT-1741: Support TLS cipher suite TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 FEAT-1741: Support server certificate validation using Certificate Revocation Lists (CRL)
8.00.xx	FEAT-2005: Release of TLS_RSA_WITH_AES_128_CBC_SHA256, also support TLS1.2 with RSA and client certificate FEAT-1996: TLS Heartbeat Extension
9.00.xx	Release of TLS as a product component
9.01.xx	FEAT-2889: Support TLS 'trusted_ca_keys' extension as specified in RFC 6066
9.02.xx	STORYC-592: TLS Server CertChain Callout in TLS Client STORYC-638: Report Invalid Server CertChain STORYC-1506: Grant access to the TLS master secret
9.03.xx	STORYC-4467: Adapt TLS to be compatible with TcpIp that only provids the ASR4.2 transmit API

Table 1-1 Component history

2 Introduction

This document describes the functionality, API and configuration of the BSW module TLS. TLS is not an AUTOSAR defined module.

Supported AUTOSAR Release*:	4.2.1+	
Supported Configuration Variants:	pre-compile	
Vendor ID:	TLS_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	TLS_MODULE_ID	255 decimal (according to ref. [3])
Module Instance:	TLS_INSTANCE_ID	103 decimal (default value, chosen by Vector)

* For the detailed functional specification please also refer to the corresponding AUTOSAR SWS.

The TLS provides secure encrypted communication over a connection oriented transport layer (here: TCP). The Tls module implements a TLS client.

2.1 Architecture Overview

The following figure shows where the TLS is located in the AUTOSAR architecture.

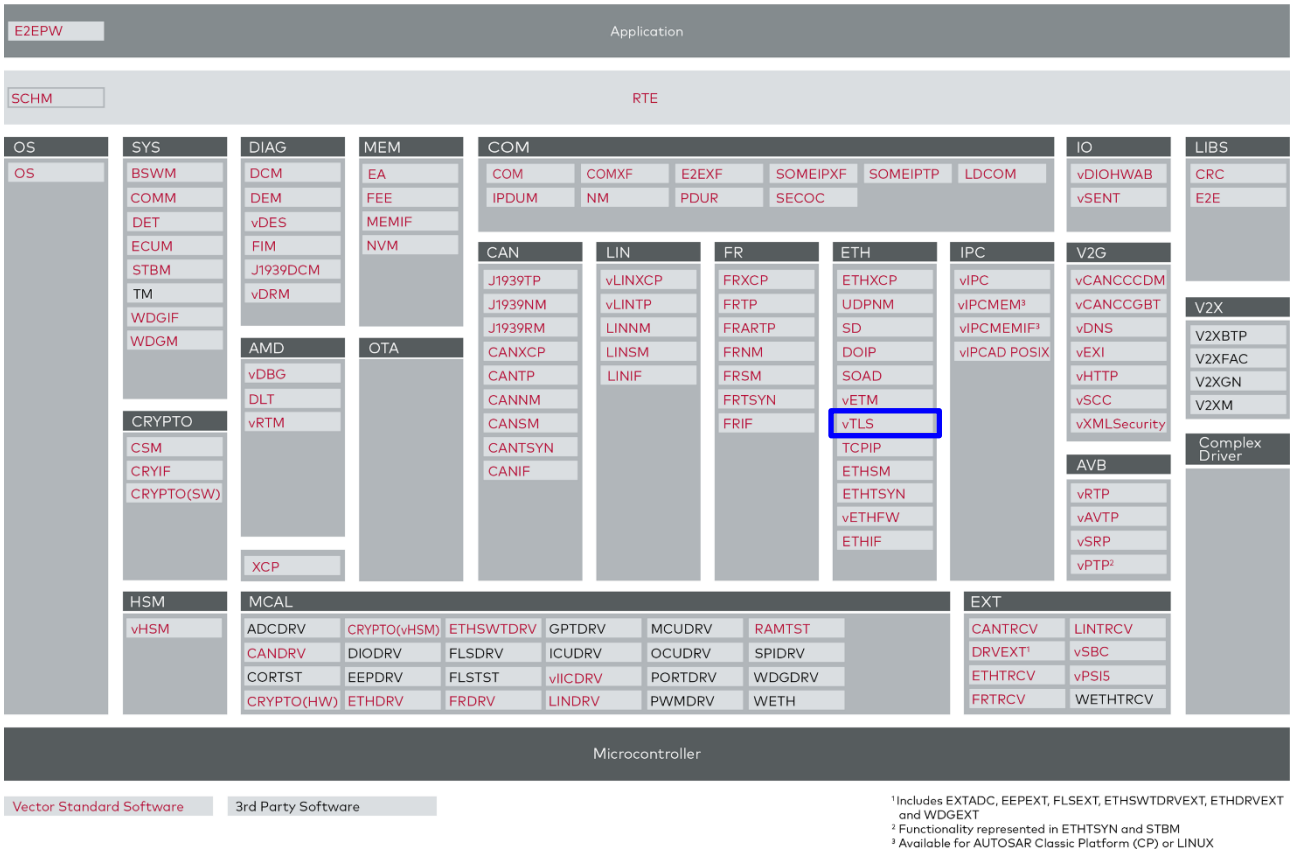


Figure 2-1 AUTOSAR 4.3 Architecture Overview

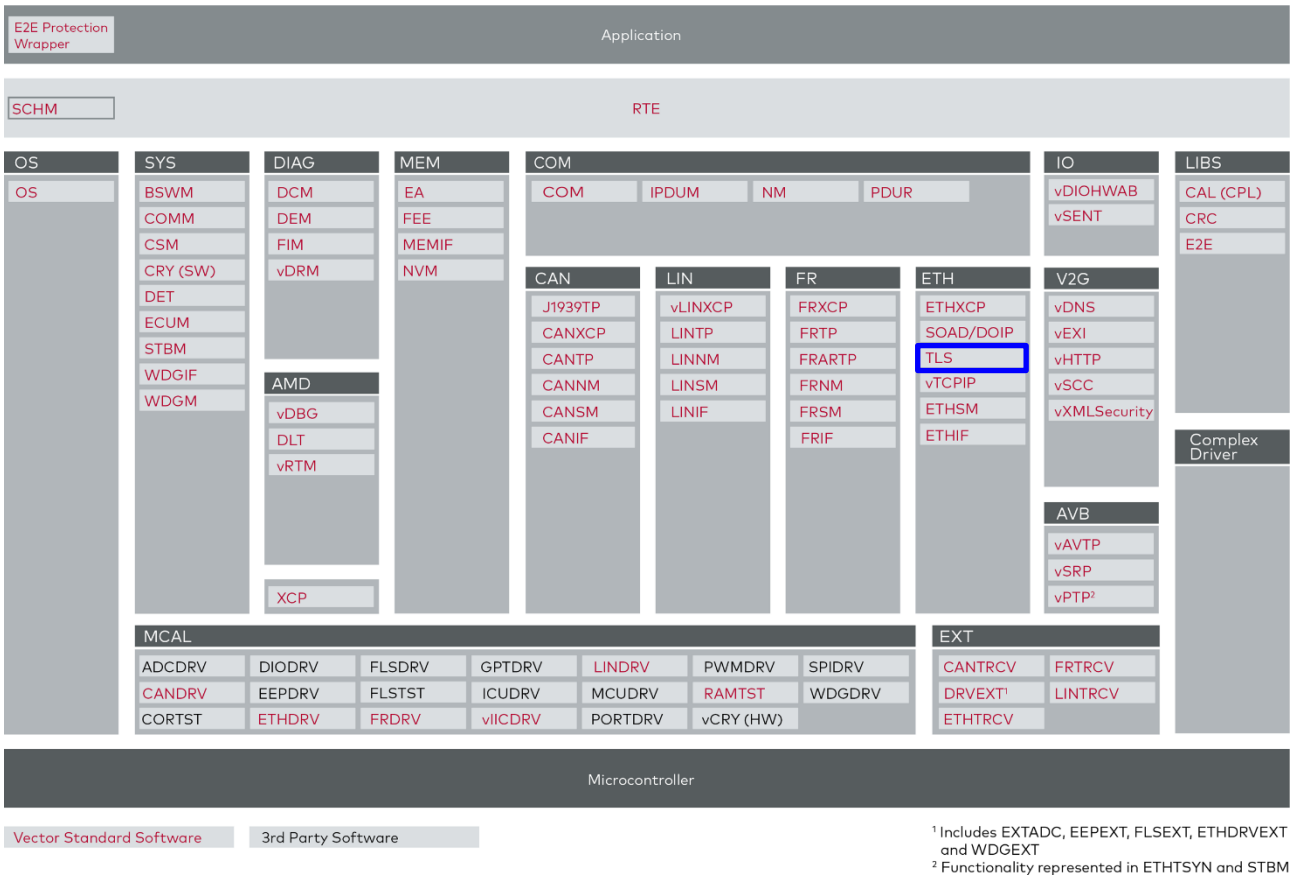


Figure 2-2 AUTOSAR 4.x Architecture Overview

The next figure shows the interfaces to adjacent modules of the TLS. These interfaces are described in chapter 4.3.8.

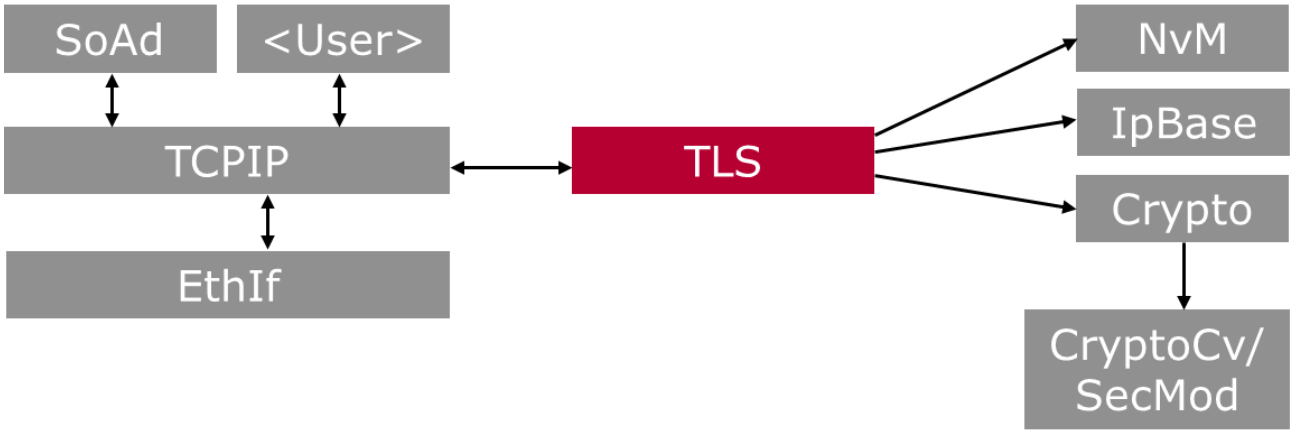


Figure 2-3 Interfaces to adjacent modules of the TLS

Applications access the services of the BSW modules indirectly through the module TcpIp (without using service ports provided by the BSW module via the RTE).

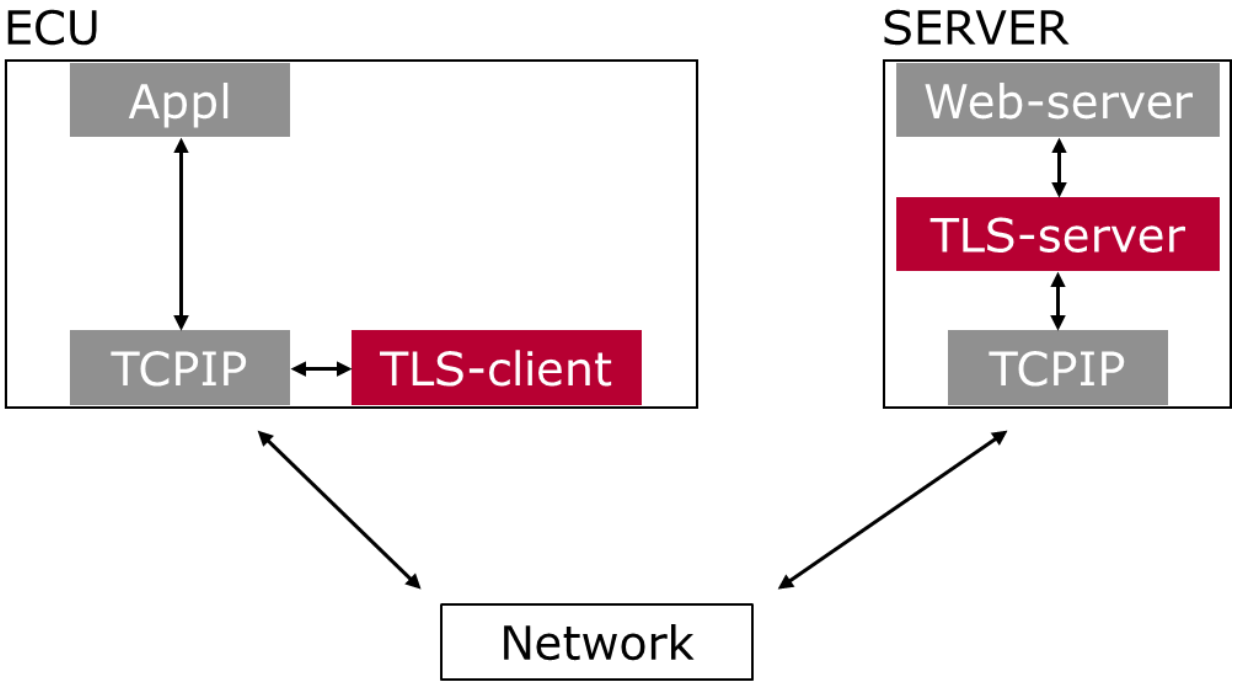


Figure 2-4 Global TLS usage architecture

3 Functional Description

3.1 Features

The TLS module supports protocol version 1.0 and 1.2. The module can be configured to support protocol version 1.0, 1.2 or both in parallel.

3.1.1 Supported Cipher Suites

The following TLS cipher suites are supported

- TLS_NULL_WITH_NULL_NULL (only for first handshake)
- TLS_RSA_WITH_NULL_SHA
- TLS_RSA_WITH_AES_128_CBC_SHA
- TLS_RSA_WITH_NULL_SHA256
- TLS_RSA_WITH_AES_128_CBC_SHA256
- TLS_ECDH_ECDSA_WITH_NULL_SHA
- TLS_ECDH_ECDSA_WITH_AES_128_CBC_SHA
- TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA
- TLS_ECDH_ECDSA_WITH_AES_128_CBC_SHA256
- TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256
- TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256

3.1.2 Support certificate validation using OCSP

TLS supports certificate validation using Online Certificate Status Protocol (OCSP) stapling V1 [12].

3.1.3 Support certificate validation using CRL

TLS supports certificate validation using Certificate Revocation Lists (CRL) according to RFC 5280 [10]. There is a global container to specify all stored CRLs (in NvM). The CRLs are parsed after module startup. When receiving a server certificate during the TLS handshake it is validated that the certificate is not listed in the matching CRL.

The current implementation only supports the validation of the server certificate using CRL, not the further received certificate chain.

The current implementation has no mechanism to update the NvM blocks that contain the CRLs but this has to be handled by other means. When the CRLs are updated during runtime, the TLS needs to be informed about this update to update its parsed information about the CRLs.

3.1.4 Support TLS 'trusted_ca_keys' extension as specified in RFC 6066

Tls supports the extension "trusted_ca_indication" as specified by RFC 6066 [11]. The indication is sent as an extension in the client_hello message. In this extension the client sends a hash of each root certificate known to the client.

There is no need for any configuration or user interaction.

3.1.5 TLS Server CertChain Callout in TLS Client

The Tls client can indicate the certificate chain received in the 'certificate' message to the socket owner, combined with the Tls validation result. This is done in an optional callback configured in the TcpIp module for a socket owner.

The indication forwards two elements:

- The complete certificate as received from the server. Each certificate is preceded by a three byte length information.
- The validation result for the certificate chain.

The socket owner then decides what to do with the certificate chain (e.g. do some further evaluations, or store a received certificate as a root certificate to the NvM). Then the socket owner can decide if Tls shall accept the certificate chain and proceed with the handshake, or if the handshake shall be canceled. Making its decision the socket owner can overrule the Tls validation result.

3.1.6 Limitations

Following describes limitations of the implementation.



Caution
The list of limitations makes no claim to be complete!

3.1.6.1 L001: Client support only

The current version of the TLS implements only a TLS client. So the module cannot act as a server and accept any incoming connection requests.

3.1.6.2 L002: Message size

The current implementation limits the maximum size of incoming messages. This is needed since messages have to be buffered completely to check the message authentication. Forwarding parts of the message without knowing its validity is too dangerous.

3.1.6.3 L005: TLS fragmentation

The current implementation of TLS does not support message fragmentation (for transmission) on TLS level. Encrypted messages have to fit into TLS internal buffers.

3.1.6.4 L007: Maximum number of certificates in the CRL

The maximum supported number of revoked certificate listed in a CRL is limited to 254 certificates. This limitation is caused by the usage of uint8 for chapter indices in the module IpBase (used for BER decoding).

3.1.6.5 L008: Record Layer message size limit negotiation

The current implementation does not support the negotiation of a maximum segment size during the connection establishment / handshake.

3.2 Initialization

The TLS is initialized by calling the `Tls_InitMemory` service and the `Tls_Init` service with the configuration as parameter. Since the TLS module currently does not support PostBuild functionality the configuration parameter used for Tls_Init can be NULL.

The Crypto module has to be initialized before the TLS module.

3.3 States

The TLS is operational after initialization.

3.4 Main Functions

The TLS has two main functions. The function Tls_MainFunction processes state changes, transmission and reception of messages and the function Tls_MainFunctionLowPrio handles the crypto calculations during the TLS handshake.

3.5 Error Handling

3.5.1 Development Error Reporting

By default, development errors are reported to the DET using the service `Det_ReportError()` as specified in [1], if development error reporting is enabled (i.e. pre-compile parameter `TLS_DEV_ERROR_DETECT==STD_ON`).

If another module is used for development error reporting, the function prototype for reporting the error can be configured by the integrator, but must have the same signature as the service `Det_ReportError()`.

The reported TLS ID is 255, the reported module Instance ID is 103.

The reported service IDs identify the services which are described in 5.4. The following table presents the service IDs and the related services:

Service ID	Service
[ServiceID]	[Service Name]
0x01	Tls_Init
0x02	Tls_RxIndication
0x03	Tls_GetVersionInfo
0x04	Tls_Connect
0x05	Tls_Close

Service ID	Service
0x06	Tls_Received
0x07	Tls_ProvideTxBuffer
0x08	Tls_TransmitTo
0x09	Tls_TcpAccepted
0x0A	Tls_TcpConnected
0x0B	Tls_TcpIpEvent
0x27	Tls_GetNvmBlockIdForUsedRootCert
0x28	Tls_RootCertWasModified
0x2B	Tls_ChangeParameter
0x2C	Tls_CrlWasModified
0x2D	Tls_GetSocket
0x2E	Tls_SetClientCertInfo
0x2F	Tls_GetMasterSecret

Table 3-1 Service IDs

The errors reported to DET are described in the following table:

Error Code	Description
[Error Code]	[Description]
0x01	Module not initialized, e.g. Tls_Init not yet finished
0x02	Invalid parameter pointer, e.g. function called with NULL_PTR
0x03	Invalid parameter, e.g. function called with invalid parameter value
0x04	Invalid socket ID, e.g. function called with invalid socket parameter
0x05	Buffer size too small, e.g. configured receive buffer is too small
0x06	Crypto library not initialized, e.g. crypto initialization not yet finished
0x07	Crypto library reported an error

Table 3-2 Errors reported to DET

3.5.2 Production Code Error Reporting

The current implementation does not support any production error reporting according to the AUTOSAR standard (using the DEM module as specified in [2]).

4 Integration

This chapter gives necessary information for the integration of the MICROSAR TLS into an application environment of an ECU.

To use the Tls module, the modules ‘TcpIp’, ‘IpBase’, ‘Crypto’ and ‘CryptoCv/SecMod’ are required, too. Therefore the headers TcpIp.h, IpBase.h and Crypto.h will be included.

4.1 Scope of Delivery

The delivery of the TLS contains the files which are described in the chapters 4.1.1 and 4.1.2:

4.1.1 Static Files

File Name	Description
`Tls.c`	Implementation
`Tls.h`	API declaration
`Tls_Cbk.h`	API call-back declaration
`Tls_Priv.h`	Component local macro and variable declaration
`Tls_Lcfg.h`	Link-time parameter configuration declaration
`Tls_Types.h`	Types header for API of Tls

Table 4-1 Static files

4.1.2 Dynamic Files

The dynamic files are generated by the configuration tool [config tool].

File Name	Description
`Tls_Cfg.h`	Pre-compile time parameter configuration
`Tls_Lcfg.c`	Link-time parameter configuration

Table 4-2 Generated files

4.2 Critical Sections

The TLS calls service functions of upper layers in order to prevent interruption of critical sections.

These service functions have to be provided (normally by the Schedule Manager) and configured accordingly.

TLS_EXCLUSIVE_AREA_0: This is the only critical section used in TLS. Currently this critical section is only used in Tls_RxIndication and Tls_VReleaseBufSegment. The critical section is used write the buffer segment and to release the buffer segment. It is required to prevent further messages to be received by TLS. Depending on the Ethernet frame reception configuration it is sufficient to lock Ethernet receive interrupt.

4.3 General integration notes

TLS offers a secure communication between two nodes. This communication has to be established using the TLS internal handshake protocol. During the handshake a lot of cryptographic calculations have to be done that consume a significant amount of time and therefore cannot be performed in a single Tls_MainFunction cycle. Certificate and message signatures have to be checked, and asymmetric en-/de-cryptions have to be done.

These calculations are executed in the context of the Tls_MainFunctionLowPrio. This function is called in a BackgroundTask. Using a background task enables the rest of the ECU to operate quite normal and have further communication on different busses. On a 32-bit CPU with 80MHz the calculations for a Handshake using an ECDSA cipher suite could take approximately 1s pure calculation time, and being interrupted all the time makes it even much slower.

4.3.1 External access to memory (e.g. XCP or Diagnostics) has to be limited

For series production ECUs memory access has to be limited so that no RAM or ROM used by TLS is accessible via read or write. Therefore calibration access (e.g. XCP), diagnostics access (e.g. UDS) and any other access has to be switched off entirely or the access to memory and variables has to be switched off. The memory used by TLS can be mapped via AUTOSAR memory mapping.



Caution
This limitation is required to ensure the security of TLS connections and private keys of certificates.

4.3.2 Access to real global time and data information

For the validation of received certificates not only the signature has to be checked, but also the valid life time of a certificate has to be checked. To be able to check times noted in the certificates, TLS needs access to an absolute time information. Therefore TLS configures a callback to get the real time and date information from the application.

4.3.3 Access to real random values

TLS needs good random values. For derivation of random values during runtime TLS uses a random function from the Crypto module. But before the Crypto random functions can be used an initial random value is needed as a seed, since the random functions in Crypto can only produce pseudo random values based on the initial seed. Therefore TLS configures a callback to get an initial seed from the application.

4.3.4 CRL configuration

To use CRLs for server certificate validation, the matching CRLs have to be configured. For each CRL a block in NVM is needed. For the configuration of the parameter '/MICROSAR/Tls/TlsGeneral/TlsCrlNvmExchData' the size of the biggest block used for the CRLs is needed.

4.3.5 Heartbeat configuration

To use the TLS heartbeat this function first has to be enabled by configuring a default heartbeat period. During runtime the usage of the heartbeat has to be enabled for each socket by using the TcpIp_ChangeParameter API with ParameterId TCPIP_PARAMID_V_TLS_HEARTBEAT_MODE and parameter value TLS_HB_PEER_NOT_ALLOWED_TO_SEND (uint8). Additionally the heartbeat period can be set by using the TcpIp_ChangeParameter API with ParameterId TCPIP_PARAMID_V_TLS_HEARTBEAT_PERIOD_S and parameter value <seconds> (uint16). TLS will then cyclically send heartbeat requests to the server and evaluate the received responses. If there is no response to the heartbeat request in the configured period, the upper layer is called using the TcpIp Event callbacks, the EventType is extended to have a value 'TCPIP_TLS_HEARTBEAT_NO_RESPONSE'.

4.3.6 TLS Connection Setup

Before communication over an encrypted TLS connection is possible, the connection has to be established. The following steps are required to setup a TLS connection (s.Figure 4-1 Get a socket for TLS and set buffer sizes, Figure 4-2Bind and connect a socket)

- Get a socket by calling TcpIp_<Up>_GetSocket
- Request the usage of TLS by calling TcpIp_ChangeParameter with ParameterId TCPIP_PARAMID_V_USE_TLS and ParamterValue 'TRUE' (Boolean)
- Set buffer sizes for TCP by calling TcpIp_ChangeParameter with ParameterIds TCPIP_PARAMID_TCP_RXWND_MAX and TCPIP_PARAMID_V_TCP_TXBUFSIZE (uint32), followed by ParameterIds TCPIP_PARAMID_V_TLS_TXBUFSIZE and TCPIP_PARAMID_V_TLS_RXBUFSIZE (uint32)
- If client authentication shall be used, information about the client certificate and key have to be provided by a call of TcpIp_Tls_SetClientCertInfo
- Optionally bind the socket by calling TcpIp_Bind
- Trigger the connection establishment by calling TcpIp_TcpConnect. As soon as the connection is established the upper layer will be informed by a call of its callback <Up_TcpConnected>

4.3.7 Dependency to TcpIp

This Tls implementation is highly dependent on the Vector component TcpIp. Tls directly interacts with the TcpIp module, while all user interaction is done via the TcpIp API. For compatibility reasons the TcpIp module version has to be at least 7.00.00.

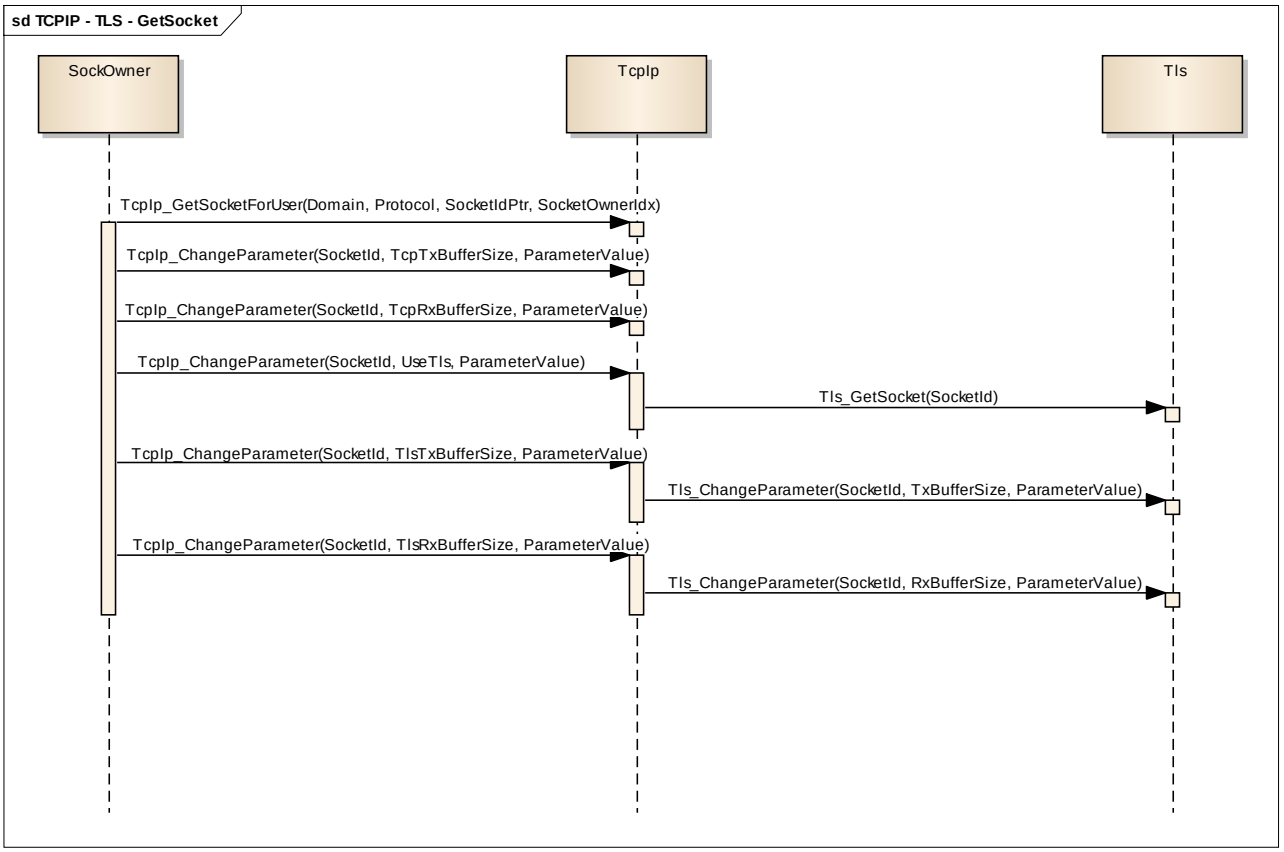


Figure 4-1 Get a socket for TLS and set buffer sizes

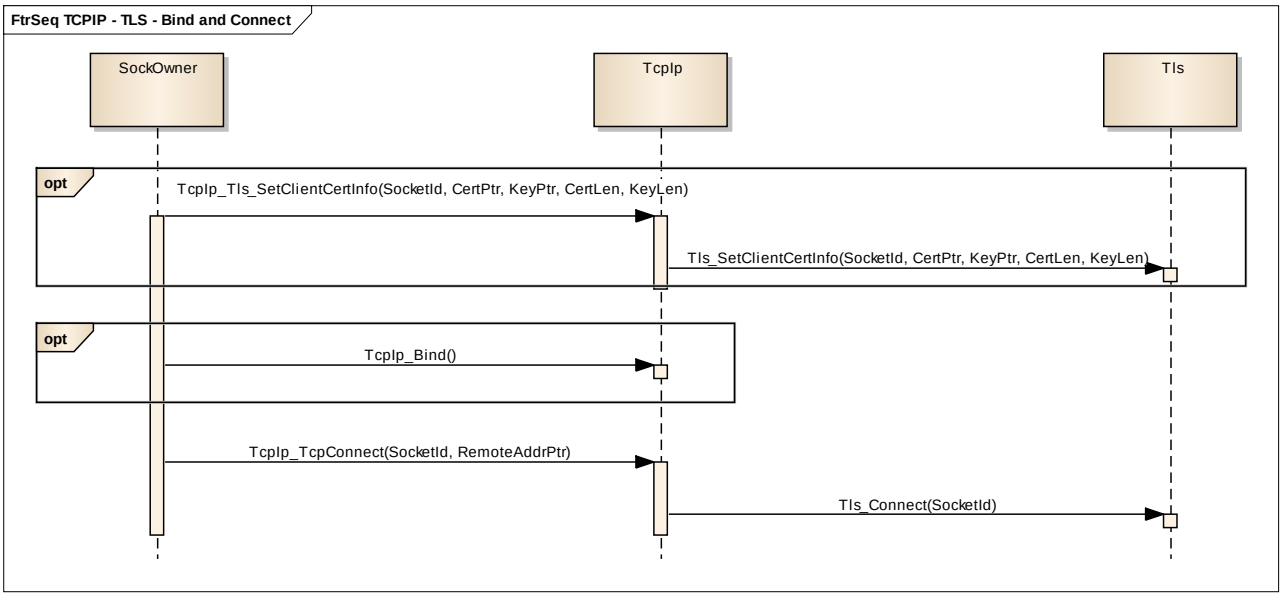


Figure 4-2 Bind and connect a socket

4.3.8 Server CertChain Callout

Tls implements a callback (TcpIp_Tcp_TlsValidationResult()) that is used to indicate the result of the certificate chain validation. This callback is called in the following cases:

- When a certificate chain could be validated successfully against one of the stored root certificates
- When a certificate chain could not be validated because no matching root certificate was found
- When a certificate chain could not be validated because the signature validation of the chain certificate against the stored root certificate failed

When the callback is issued, the socket owner can do further investigations on the certificates. Then the socket owner can overrule the validation result and determine whether the TLS handshake shall be continued or aborted, or it can just accept the validation result and leave its value as set by Tls.

The validation result can have the following values:

- TCPIP_TLS_VALIDATION_OK
- TCPIP_TLS_VALIDATION_UNKNOWN_CA
- TCPIP_TLS_VALIDATION_LAST_SIGN_INVALID
- TCPIP_TLS_VALIDATION_REFUSED_BY_OWNER

4.3.9 Access to the TLS master secret

By using the API Tls_GetMasterSecret(), access to the used master secret is granted. This feature can be enabled in the configuration. For security reasons the API is disabled by default. If it is enabled, a warning will be displayed in the configurator.



Caution
The API Tls_GetMasterSecret() may never be enabled for production code.

The API Tls_GetMasterSecret() is not wrapped by a corresponding TcpIp API but has to be called directly by the socket owner. To have access to the function declaration the socket owner has to include the header file ‘Tls.h’.

5 API Description

The API of the TLS module is nearly identical to the API of the TCPIP module defined in [4]. This is to make the TLS as transparent as possible, so that the user does not need to behave in different ways depending on the used transport layer.

5.1 TLS usage via the TcpIp module

There are different kinds of functions in TLS.

5.1.1 General functions

Functions that exist for most BSW modules (Init, InitMemory, MainFunction, MainFunctionLowPrio, GetVersionInfo). These functions are called directly by other modules (e.g. by the RTE or the BswM).

5.1.2 Standard communication functions

There are Functions that are similar to the TcpIp communication APIs (GetSocket, Bind, Connect, Transmit, Received, Close, ChangeParameter). These functions are called by TcpIp after the socket owner called the corresponding TcpIp APIs.

5.1.3 TLS specific functions

There are Functions only for TLS (Tls_RootCertWasModified, Tls_GetNvmBlockIdForUsedRootCert, Tls_SetClientCertInfo). These functions are called via TcpIp by prepending a ‘TcpIp_’ to the API name (e.g. TcpIp_Tls_RootCertWasModified). TcpIp will forward these functions directly to TLS.

5.2 Type Definitions

The types defined by the TLS are described in this chapter.

Type Name	C-Type	Description	Value Range
`Tls_ConfigType`	uint8	Config type used for post-build configuration	Byte array with implementation specific configuration
`TcpIp_ParamIdType (extension of the TCPIP type)`	uint8	Used to identifiy parameters changed via the ChangeParameter API	`TCPIP_PARAMID_V_USE_TLS` Configure TcpIp to use TLS for this socket
			`TCPIP_PARAMID_V_TLS_TXBUFSIZE` Configure the transmit buffer size for TLS
			`TCPIP_PARAMID_V_TLS_RXBUFSIZE` Configure the receive buffer size for TLS
			`TCPIP_PARAMID_V_TLS_SELECT_OCSP_REQEST`

Type Name	C-Type	Description	Value Range
			Specify OCSP requests for the connection
			TCPIP_PARAMID_V_TLS_HEARTBEAT_MODE Specify heartbeat mode for the connection
			TCPIP_PARAMID_V_TLS_HEARTBEAT_PERIOD_S Specify heartbeat period for the connection
TcpIp_TcpIpEventType (extension of the TCPIP type)	uint8	Used to indicate events via <User>_TcpIpEvent	TCPIP_TLS_HEARTBEAT_NO_RESPONSE No heartbeat response has been received in time after sending a heartbeat request
Tls_SocketType	struct	Used to define the domain (IPv4 or IPv6)	TCPIP_AF_UNDEFINED 0x0000U TCPIP_AF_INET 0x0002U TCPIP_AF_INET6 0x001CU
Tls_SockAddrIn6Type	struct	Used to define the domain, port and address	Domain IPv4 or IPv6
			Port Port Number
			Addr IP address
Tls_PbufType	struct	Used to store data in distributed memory	payload pointer to data
			totLen total length in bytes
			Addr block length in bytes
Tls_SocketIdType	uint8	Used to identify sockets	Generated identifier

Table 5-1 Type definitions

5.3 Interrupt Service Routines provided by TLS

None

5.4 Services provided by TLS

TLS does not offer any API to the user directly but only through TcpIp.

5.4.1 Tls_InitMemory

Prototype
`void Tls_InitMemory (void)`

Parameter	
void	none
Return code	
void	none
Functional Description	
Initializes global variables of the TLS module.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is non-reentrant. > This function is an extension to AUTOSAR. It has to be called before any other calls to the module.	
Call context	
> task level	

Table 5-2 Tls_InitMemory

5.4.2 Tls_Init

Prototype
`void Tls_Init (const Tls_ConfigType *CfgPtr)`
Parameter
CfgPtr [in] — Pointer to module configuration
Return code
void — none
Functional Description
Initializes the TLS module.
Particularities and Limitations
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is non-reentrant. > Has to be called before usage of the module.
Call context
> task level

Table 5-3 Tls_Init

5.4.3 Tls_GetVersionInfo

Prototype
`void Tls_GetVersionInfo (Std_VersionInfoType *VersionInfoPtr)`

Parameter	
VersionInfoPtr [out]	pointer for version info
Return code	
void	none
Functional Description	
Returns the version information.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is non-reentrant.	
Call context	
> interrupt or task level	

Table 5-4 Tls_GetVersionInfo

5.4.4 Tls_GetNvmBlockIdForUsedRootCert

Prototype	
`Std_ReturnType` **`Tls_GetNvmBlockIdForUsedRootCert`** `(Tls_SocketIdType SocketId, NvM_BlockIdType *RootCertBlockIdPtr)`	
Parameter	
SocketId [in]	Socket ID
RootCertBlockIdPtr [out]	Pointer to the block ID
Return code	
Std_ReturnType	> E_OK Block ID can be provided > E_NOT_OK request failed
Functional Description	
Get the NVM block ID of the root certificate used for a TLS connection.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is non-reentrant.	
Call context	
> task level	

Table 5-5 Tls_GetNvmBlockIdForUsedRootCert

5.4.5 Tls_RootCertWasModified

Prototype	
`Std_ReturnType` **`Tls_RootCertWasModified`** `(NvM_BlockIdType NvmBlockId)`	
Parameter	
NvmBlockId [in]	Block ID of the NVM block that has changed
Return code	
Std_ReturnType	> E_OK Block is used as a root certificate > E_NOT_OK request failed, block is not used as a root certificate
Functional Description	
Inform the TLS that a root cert has been updated in NVM.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is non-reentrant.	
Call context	
> task level	

Table 5-6 Tls_RootCertWasModified

5.4.6 Tls_CrlWasModified

Prototype	
`Std_ReturnType` **`Tls_CrlWasModified`** `(NvM_BlockIdType NvmBlockId)`	
Parameter	
NvmBlockId [in]	Block ID of the NVM block that has changed
Return code	
Std_ReturnType	E_OK Block is used as a CRL E_NOT_OK request failed, block is not used as a CRL
Functional Description	
Inform the TLS that a CRL has been updated in NVM.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is non-reentrant.	
Call context	
task level	

Table 5-7 Tls_RootCertWasModified

5.4.7 Tls_MainFunction

Prototype	
`void Tls_MainFunction (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Handle the handshake and other state dependent actions. This function handles the different states to establish TLS connections, parses stored root certificates and CRLs, transmitts data and forwards data to the upper layer.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is non-reentrant.	
Call context	
> task level	

Table 5-8 Tls_MainFunction

5.4.8 Tls_MainFunctionLowPrio

Prototype	
`void Tls_MainFunctionLowPrio (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Main function to do all crypto computations. This function shall be called from a lower priority task. The execution time can be very long in case of complex crypto calculations.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is non-reentrant.	
Call context	
> task level	

Table 5-9 Tls_MainFunction

5.4.9 Tls_GetSocket

Prototype	
`Std_ReturnType` **`Tls_GetSocket`** `(Tls_SocketIdType SocketId)`	
Parameter	
SocketId [in]	Socket ID
Return code	
Std_ReturnType	E_OK The request has been accepted E_NOT_OK The request has not been accepted: no free socket
Functional Description	
Request the preparation of one connection for a socket.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is non-reentrant.	
Call context	
task level	

Table 5-10 Tls_GetSocket

5.4.10 Tls_ChangeParameter

Prototype	
`Std_ReturnType` **`Tls_ChangeParameter`** `(Tls_SocketType SocketId, TcpIp_ParamIdType ParameterId, uint8 *ParameterValue)`	
Parameter	
SocketId [in]	Socket ID
ParameterId [in]	Parameter ID
ParameterValue [in]	Pointer to parameter value
Return code	
Std_ReturnType	E_OK parameter changed E_NOT_OK parameter change failed
Functional Description	
Set a parameter to change the configuration of a socket / connection.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is non-reentrant.	
Call context	
task level	

Table 5-11 Tls_ChangeParameter

5.4.11 Tls_SetClientCertInfo

Prototype	
`Std_ReturnType **Tls_SetClientCertInfo** (Tls_SocketIdType SocketId, uint8 *CertPtr, uint8 *KeyPtr, uint16 CertLen, uint16 KeyLen)`	
Parameter	
SocketId [in]	Socket ID
CertPtr [in]	Pointer to client certificate (RAM block)
KeyPtr [in]	Pointer to client private key (RAM block)
CertLen [in]	Length of client certificate
KeyLen [in]	Length of client private key
Return code	
Std_ReturnType	> E_OK cert info could be set > E_NOT_OK cert info could NOT be set
Functional Description	
Set the client certificate and key block ids (necessary for client authentication).	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is non-reentrant.	
Call context	
> task level	

Table 5-12 Tls_SetClientCertInfo

5.4.12 Tls_Close

Prototype	
`Std_ReturnType **Tls_Close** (Tls_SocketIdType SocketId, boolean Abort)`	
Parameter	
SocketId [in]	Socket ID
Abort	> TRUE: connection will immediately be terminated by TCP, and TLS can reset all its connection variables immediatelly > FALSE: connection will be terminated after performing a regular connection termination
Return code	
Std_ReturnType	> E_OK Close trigger could be accepted > E_NOT_OK Close trigger could NOT be accepted
Functional Description	
Close TLS connection / socket.	

Particularities and Limitations
> Service ID: see table 'Service IDs'
> This function is synchronous.
> This function is non-reentrant.
Call context
> task level

Table 5-13 Tls_Close

5.4.13 Tls_ProvideTxBuffer

Prototype	
`BufReq_ReturnType` **`Tls_ProvideTxBuffer`** `(Tls_SocketIdType SocketId, Tls_PbufType **PbufPtr, uint32 Length)`	
Parameter	
SocketId [in]	Socket ID
PbufPtr [out]	Pointer for the data
Length [in]	Data length in bytes
Return code	
BufReq_ReturnType	> BUFREQ_OK Buffer could be provided > BUFREQ_E_NOT_OK Provide buffer failed
Functional Description	
Request tx buffer for sending data.	
Particularities and Limitations	
> Service ID: see table 'Service IDs'	
> This function is synchronous.	
> This function is non-reentrant.	
Call context	
> task level	

Table 5-14 Tls_ProvideTxBuffer

5.4.14 Tls_TransmitTo

Prototype	
`Std_ReturnType` **`Tls_TransmitTo`** `(Tls_SocketIdType SocketId, const Tls_PbufType *PbufPtr)`	
Parameter	
SocketId [in]	Socket ID
PbufPtr [in]	Pointer to the data

Return code	
Std_ReturnType	> E_OK Data transmission request could be accepted > E_NOT_OK Data transmission request could NOT be accepted
Functional Description	
Trigger sending of the data or free the requested buffer.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is non-reentrant.	
Call context	
> task level	

Table 5-15 Tls_TransmitTo

5.4.15 Tls_Received

Prototype	
`Std_ReturnType Tls_Received (Tls_SocketIdType SocketId, uint32 Length)`	
Parameter	
SocketId [in]	Socket ID
Length [in]	Length in bytes
Return code	
Std_ReturnType	> E_OK Data reception acknowledged > E_NOT_OK Data reception acknowledge failed
Functional Description	
Acknowledge received data (to the stack).	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is non-reentrant.	
Call context	
> task level	

Table 5-16 Tls_Received

5.4.16 Tls_GetMasterSecret

Prototype	
`Std_ReturnType, TLS_CODE) Tls_GetMasterSecret(Tls_SocketIdType SocketId, uint8 **MsDataPtr, uint8 *MsLenPtr)`	
Parameter	
SocketId [in]	Socket ID
MsDataPtr [out]	Pointer to the master secret
MsLenPtr [out]	Pointer where the length of the master secret shall be written to
Return code	
Std_ReturnType	> E_OK Access to the master secret was successful > E_NOT_OK Access to the master secret failed
Functional Description	
Get the master secret for a TLS connection.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is non-reentrant. > This function may not be used in production code since this would be a security issue.	
Call context	
> TASK	

Table 5-17 Tls_GetMasterSecret

5.5 Services used by TLS

In the following table services provided by other components, which are used by the TLS are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
DET (optional)	`Det_ReportError`

IpBase	`IpBase_Copy`
	`IpBase_BerInitWorkspace`
	`IpBase_BerGetElement`
	`IpBase_StrCmpLen`
	`IpBase_ReadByteInPbuf`
	`IpBase_CopyPbuf2String`
	`IpBase_ConvString2Int`

AsrTcpIp	`TcpIp_Tcp_TlsTxConfirmation`
	`TcpIp_Tcp_TlsValidationResult`
	`TcpIp_Tcp_TlsConnected`
	`TcpIp_Tcp_TlsEvent`
	`TcpIp_GetSizeOfSocketDyn`

Component	API
	TcpIp_GetSizeOfSocketUdpDyn TcpIp_TcpTransmitInternal TcpIp_Tcp_TlsRxIndication TcpIp_Close TcpIp_Tcp_TlsReceived
Crypto	Crypto_CheckInit Crypto_StirRNG Crypto_GenerateEcdsaSignature Crypto_HmacSha256Init Crypto_HmacSha256Encode Crypto_HmacSha256End Crypto_HmacMd5Init Crypto_HmacMd5Encode Crypto_HmacMd5End Crypto_initVerifyRSAMD5_V15 Crypto_updateVerifyRSAMD5_V15 Crypto_finalizeVerifyRSAMD5_V15 Crypto_ValidateRsaSignature Crypto_ValidateEcdsaSignature
CryptoCv	esl_getBytesRNG esl_initSHA256 esl_initSHA1 esl_initMD5 esl_updateSHA256 esl_updateSHA1 esl_updateMD5 esl_initWorkSpaceHeader esl_initEncryptRSA_V15 esl_encryptRSA_V15 esl_initEncryptAES128 esl_encryptAES128 esl_finalizeEncryptAES128 esl_initDecryptAES128 esl_decryptAES128 esl_initHashMACSHA1 esl_updateHashMACSHA1 esl_finalizeHashMACSHA1 esl_initGenerateKeyEcP_prim esl_getLengthOfEcPpublicKey_comp esl_generateKeyEcP_prim esl_initGenerateSharedSecretDHEcP_prim esl_generateSharedSecretDHEcP_prim esl_finalizeMD5 esl_finalizeSHA1 esl_finalizeSHA256 esl_initEncryptGCM esl_updateAuthDataEncryptGCM esl_updatePlaintextEncryptGCM esl_finalizeDecryptGCM esl_initVerifyRSASHA1_V15 esl_updateVerifyRSASHA1_V15

Component	API
	esl_finalizeVerifyRSASHA1_V15

Table 5-18 Services used by the TLS

5.6 Callback Functions

This chapter describes the callback functions that are implemented by the TLS and can be invoked by other modules. The prototypes of the callback functions are provided in the header file Tls_Cbk.h by the TLS.

5.6.1 Tls_TcpConnected

Prototype	
void **Tls_TcpConnected** (Tls_SocketIdType SocketId)	
Parameter	
SocketId [in]	Socket ID
Return code	
void	none
Functional Description	
Tls callback, remote side of this socket accepted a connection request.	
Particularities and Limitations	
> This function is synchronous. > This function is non-reentrant.	
Call context	
> task level	

Table 5-19 Tls_TcpConnected

5.6.2 Tls_TcpIpEvent

Prototype	
void **Tls_TcpIpEvent** (Tls_SocketIdType SocketId, IpBase_TcpIpEventType Event)	
Parameter	
SocketId [in]	Socket ID
Event [in]	Event that occurred in TcpIp
Return code	
void	none
Functional Description	
TcpIp event callback, TCP issued an event.	

Particularities and Limitations
> Service ID: see table 'Service IDs'
> This function is synchronous.
> This function is non-reentrant.
Call context
> task level

Table 5-20 Tls_TcpIpEvent

5.6.3 Tls_RxIndication

Prototype	
`void Tls_RxIndication (Tls_SocketIdType SocketId, uint8 *DataPtr, uint16 DataLen)`	
Parameter	
SocketId [in]	Socket ID
DataPtr [in]	Pointer to the received data
DataLen [in]	Length of the received data
Return code	
void	none
Functional Description	
Receive Indication Callback.	
Particularities and Limitations	
> Service ID: see table 'Service IDs'	
> This function is synchronous.	
> This function is non-reentrant.	
Call context	
> task level	

Table 5-21 Tls_RxIndication

6 Configuration

6.1 Configuration Variants

The TLS supports the configuration variants

- VARIANT-PRE-COMPILE

The configuration classes of the TLS parameters depend on the supported configuration variants. For their definitions please see the TLS_bswmd.arxml file.

6.2 Configuration with DaVinci Configurator Pro

For a detailed description of the parameters and possible values see Help in DaVinci Configurator Pro.

7 Glossary and Abbreviations

7.1 Glossary

Term	Description
DaVinci Configurator Pro	Generation tool for CANbedded and MICROSAR components (MICROSAR 4)
Platform	Hardware including Host and Communication Controller (might also be integrated in Host) on which the communication stack is implemented.

Table 7-1 Glossary

7.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BER	Basic Encoding Rules
BSW	Basis Software
CRL	Certificate Revocation List
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECU	Electronic Control Unit
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
RTE	Runtime Environment
SWS	Software Specification
TCP	Transmission Control Protocol
TLS	Transport Layer Security
UDP	User Datagram Protocol

Table 7-2 Abbreviations

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com

© 2018 Vector Informatik GmbH